

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática
CI-0116 Análisis de Algoritmos y Estructuras de Datos

Tarea Programada 1

Tercera entrega: Análisis empírico de rendimiento

Autores:

Marina Castro Peralta C31886
Emanuel González Chaves C03361

Profesor: Braulio José Solano Rojas

I Ciclo, 2026

1. Resumen

Este documento presenta el análisis empírico de siete (7) implementaciones del modelo Función (arreglo asociativo llave-valor) desarrolladas en Python como parte de la Tarea Programada 1. Se midieron las operaciones Assign, Lookup, Unassign, Print, Done y el uso de memoria en tres tamaños de entrada ($n = 100$, $50\,000$, $1\,000\,000$), con 10 corridas por experimento. Las estructuras evaluadas son: Lista Ordenada Dinámica y Estática, Tabla Hash Abierta (sin redistribución), Árbol Binario de Búsqueda con punteros y con vector heap, y Trie con punteros y con arreglos.

Los resultados confirman las predicciones teóricas: el ABB con punteros ofrece el mejor balance general ($\approx 15\ \mu\text{s}$ por operación a $n = 1\ \text{M}$), la Lista Ordenada y la Tabla Hash sin redistribución son impracticables a partir de $n \approx 5\,000$ y $n \approx 50\,000$, respectivamente, debido a su comportamiento cuadrático, y el Trie ofrece búsquedas constantes, pero con un costo de memoria prohibitivo ($O(n \cdot L)$) a grandes tamaños.

2. Especificación del Modelo Función

Una función F es un arreglo asociativo finito de pares (llave, valor) en el que las llaves son únicas. Formalmente, $F : K \rightarrow V$, donde K es el dominio de llaves (hileras de hasta 20 caracteres alfanuméricos) y V es el dominio de valores (también hileras). La relación es funcional: para cada llave existe a lo sumo un valor asociado.

2.1 Operaciones del modelo

La interfaz abstracta Función (archivo funcion.py) define seis operaciones que toda implementación concreta debe proporcionar. Su semántica es la siguiente:

Operación	Firma	Semántica
asigne	<code>asigne(llave, valor) → None</code>	Si la llave existe, actualiza su valor; si no, agrega el par (llave, valor).
elimine	<code>elimine(llave) → None</code>	Si la llave existe, elimina el par asociado; de lo contrario, no realiza ninguna acción.
obtenga	<code>obtenga(llave) → valor None</code>	Retorna el valor asociado a la llave; None si la llave no existe.
limpie	<code>limpie() → None</code>	Elimina todas las relaciones y deja la función vacía.
llaves	<code>llaves() → lista[K]</code>	Retorna la lista de llaves presentes en la función.
imprima	<code>imprima() → None</code>	Imprime en la salida estándar todos los pares (llave, valor) almacenados.

Se asume el invariante de unicidad: en todo momento las llaves almacenadas son distintas dos a dos. Cada estructura concreta debe preservar este invariante mediante las operaciones

asigne y elimine. Además, se implementan los métodos especiales `__len__`, `__getitem__`, `__str__` y `__del__` para integrarse con la sintaxis nativa de Python.

3. Especificación de las cuatro estructuras genéricas

Las siete implementaciones concretas se agrupan en cuatro familias de estructuras de datos. A continuación se describe cada familia, sus invariantes y la complejidad teórica de sus operaciones principales.

3.1 Lista Ordenada

Una secuencia de pares (llave, valor) mantenida en orden ascendente por la llave. La búsqueda puede aprovechar el orden (binaria en arreglos, secuencial en listas enlazadas), y las modificaciones requieren reubicar el par para mantenerlo. Costo asintótico: Lookup: $O(\log n)$ o $O(n)$; Assign: $O(n)$; Unassign: $O(n)$. Memoria $O(n)$.

3.2 Tabla Hash con encadenamiento abierto

Un arreglo de m cubetas donde cada cubeta es una lista de pares cuya llave produce el mismo índice bajo una función hash $h : K \rightarrow \{0, \dots, m-1\}$. Esta tarea utiliza la función polinomial $h(s) = (\sum s[i] \cdot p^{(L-1-i)}) \bmod m$ con $p = 31$. Cuando el factor de carga $\lambda = n/m$ supera un umbral (`UMBRAL_CARGA` = 0.75 en la implementación principal), m se duplica y todos los pares se reinsertan (redistribución). Costo asintótico esperado bajo un hash uniforme: Assign, Lookup y Unassign $O(1)$ amortizado. Memoria $O(n + m)$.

3.3 Árbol Binario de Búsqueda (ABB)

Un árbol binario en el que cada nodo contiene un par (llave, valor) y satisface la propiedad de búsqueda: las llaves del subárbol izquierdo son menores que la llave del nodo y las del subárbol derecho son mayores que la llave del nodo. Las operaciones recorren el árbol desde la raíz, comparando las llaves, con un costo proporcional a la altura del árbol. En el caso promedio (inserciones aleatorias), la altura es $O(\log n)$; en el peor caso (inserciones ordenadas), es $O(n)$.

3.4 Trie (árbol de prefijos)

Un árbol enraizado en el que cada arista representa un carácter y cada camino desde la raíz hasta un nodo marcado codifica una llave. El valor se almacena en el nodo terminal. El costo de cada operación es $O(L)$, donde L es la longitud de la llave, y es independiente del número total de llaves almacenadas. Memoria $O(n \cdot L)$ en el peor de los casos, reducida cuando las llaves comparten prefijos.

4. Especificación de las siete estructuras finales

Cada familia genérica admite varias implementaciones. La tabla 4.1 resume los detalles de implementación; los párrafos siguientes amplían las particularidades relevantes.

4.1 Tabla resumen

Estructura	Familia	Detalles de implementación
LO dinámica	Lista Ordenada	Lista enlazada de nodos {llave, valor, siguiente}. Inserción ordenada lineal.
LO estática	Lista Ordenada	Arreglo de capacidad fija ($n = 100$). Búsqueda binaria: $O(\log n)$; inserción con desplazamiento: $O(n)$.
Tabla Hash (sin redistrib.)	Tabla Hash	$m = 11$ fijo, sin redistribución. Variante para evidenciar el efecto del factor de carga.
ABB punteros	ABB	Nodos con referencias explícitas a la izquierda y a la derecha. Sin balanceo automático.
ABB vector heap	ABB	Diccionario {índice: Par} con índices de tipo heap ($izq = 2i+1$, $der = 2i+2$).
Trie punteros	Trie	Cada nodo guarda un diccionario {carácter: hijo}, que está presente sólo si el carácter aparece.
Trie arreglos	Trie	Cada nodo guarda un arreglo de 62 entradas (una por cada carácter alfanumérico).

4.2 Particularidades relevantes

La Lista Ordenada estática tiene una capacidad fija de 100 entradas; al alcanzar el límite, la operación lanza una excepción de tipo `IndexError`. Esto explica que esta estructura sólo se mida para $n = 100$ y para tamaños menores.

La Tabla Hash sin redistribución acepta un parámetro de constructor que desactiva la duplicación automática del arreglo. Sirve únicamente para demostrar empíricamente la dependencia entre el factor de carga y el tiempo de acceso; en producción siempre se usaría la variante con redistribución (implementada en el menú interactivo).

El ABB vector heap representa los nodos no mediante referencias, sino por su índice en un arreglo virtual; en la implementación se utiliza un diccionario {índice: Par} para tolerar huecos. Esta representación elimina la indirección de los punteros explícitos, pero introduce un costo de hash de enteros que crece exponencialmente con la profundidad del árbol.

El Trie de arreglos reserva 62 posiciones por nodo (26 minúsculas + 26 mayúsculas + 10 dígitos), independientemente de cuántos caracteres se utilicen realmente. El Trie de punteros (con diccionario) instancia sólo las aristas usadas. Ambas variantes mantienen la propiedad estructural del Trie y solo difieren en el almacenamiento de los hijos.

5. Metodología

5.1 Operaciones medidas

Para cada estructura y cada tamaño n se midieron seis cantidades:

- Assign — tiempo promedio (μs) de una inserción, sobre n inserciones desde cero.
- Lookup — tiempo promedio (μs) de una búsqueda exitosa.
- Unassign — tiempo promedio (μs) para una eliminación. Para $n > 10\,000$, se construyó la estructura una vez y se cronometrarón el borrado y la reinserción de una muestra de 1 000 llaves por corrida.
- Print — tiempo total (s) de la operación `imprima()` con la salida redirigida a `/dev/null`.
- Done — tiempo total (s) del destructor (`del e`).
- Memoria — KB en uso por la estructura, medidos con `tracemalloc` al final del poblado.

5.2 Ambiente experimental

Parámetro	Valor
Lenguaje	Python 3.14 (intérprete CPython)
Gestor de paquetes	uv
Cronómetro	<code>time.perf_counter()</code> (resolución sub- μs)
Medición de memoria	<code>tracemalloc</code> (memoria pedida al asignador)
Corridas por experimento	10 (media y desviación estándar)
Generador de llaves	alfanumérico, largo 3–20 caracteres (<code>random.choices</code>)
Semilla	no fijada (cada corrida con datos independientes)

5.3 Limitaciones y omisiones

Tres estructuras presentan límites prácticos que obligaron a omitir experimentos en tamaños grandes:

Estructura	Límite usado	Razón
Lista Ordenada (din/est)	$n \leq 5\,000$	Inserción $O(n) \Rightarrow$ construcción $O(n^2)$; a $n = 50\,000$, tomaría minutos por corrida.
Trie (ptr/arr)	$n \leq 50\,000$	Genera $O(n \cdot L)$ nodos en Python; con $n = 1\,\text{M}$ consume varios GB de RAM y provoca swap o terminación.
Hash sin redistribución	$n \leq 50\,000$	Factor de carga creciente $\Rightarrow$ inserción $O(n/m)$ y construcción $O(n^2/m)$; impráctico a $n = 1\,\text{M}$.

6. Las siete comparaciones requeridas

Para cada pareja se reportan una tabla de tiempos para los tamaños disponibles, la gráfica correspondiente y un análisis breve. La descripción completa de los datos se encuentra en el archivo resultados_benchmark.csv.

6.1 LO dinámica vs. LO estática

Estructura (n=100)	Assign μ s	Lookup μ s	Unassign μ s	Memoria KB
LO dinámica	11.85	9.44	5.14	17.4
LO estática	25.50	5.60	23.54	1.06

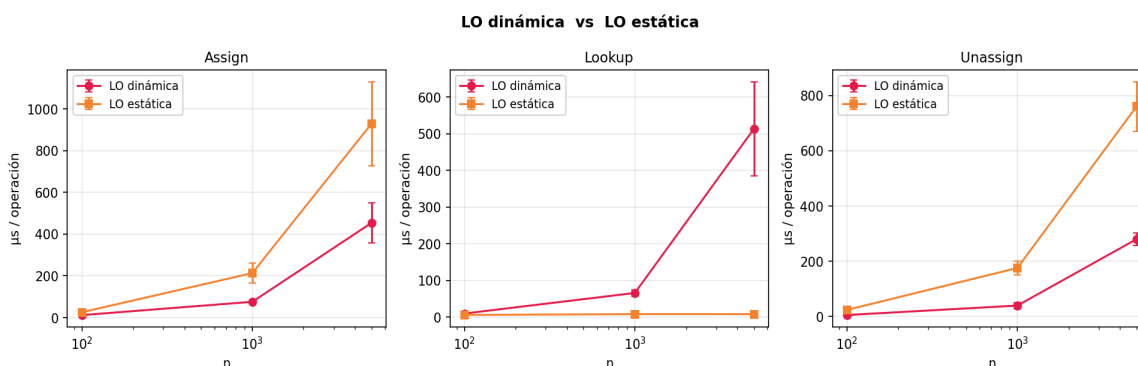


Figura 1. Comparación de LO dinámica vs LO estática en $n = 100$, $1\,000$ y $5\,000$.

La LO estática gana en Lookup porque utiliza búsqueda binaria $O(\log n)$ sobre un arreglo contiguo, mientras que la dinámica debe recorrer una lista enlazada. Sin embargo, la estática pierde en Assign y Unassign debido al desplazamiento explícito de los elementos del arreglo, mientras que la dinámica solo modifica los punteros. En memoria, la estática es 16× más eficiente (1 KB vs 17 KB a $n = 100$) porque almacena los pares contiguos sin la sobrecarga de los nodos enlazados de Python.

6.2 ABB punteros vs. ABB vector heap

Estructura	n	Assign μ s	Lookup μ s	Unassign μ s	Memoria KB
ABB punteros	100	3.18	1.19	2.30	18.1
ABB vector heap	100	2.72	2.81	4.71	13.3
ABB punteros	50 000	7.41	5.86	12.06	8 984
ABB vector heap	50 000	28.55	38.75	31.61	8 410
ABB punteros	1 000 000	14.98	13.80	30.30	179 687
ABB vector heap	1 000 000	31.47	29.35	30.99	159 055

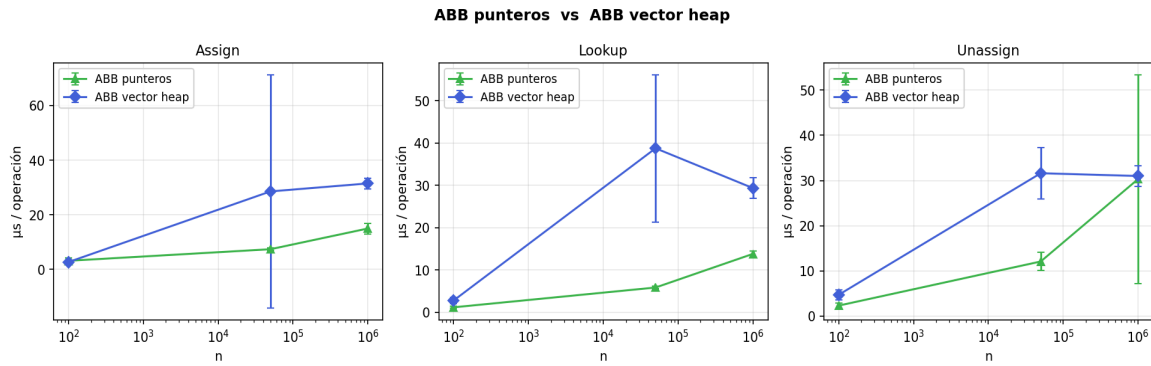


Figura 2. ABB punteros vs ABB vector heap ($n = 100, 50\text{ K}, 1\text{ M}$).

El ABB de punteros gana claramente en Lookup y Assign a partir de $n = 50\,000$ ($2\times$ a $4\times$ más rápido). La causa es que el vector heap utiliza un diccionario {índice: Par} con índices que crecen exponencialmente ($2i+1, 2i+2$): para una rama de profundidad d se manejan claves del orden de 2^d , lo que penaliza el hash interno de Python. En contraste, el ABB de punteros mantiene un costo proporcional a la profundidad lógica del árbol, no a la magnitud del índice.

6.3 Trie punteros vs. Trie arreglos

Estructura	n	Assign µs	Lookup µs	Unassign µs	Memoria KB
Trie punteros	100	6.86	1.24	8.61	266
Trie arreglos	100	28.10	5.03	171.23	884
Trie punteros	50 000	17.19	3.49	11.39	124 619
Trie arreglos	50 000	59.75	9.94	176.03	396 087

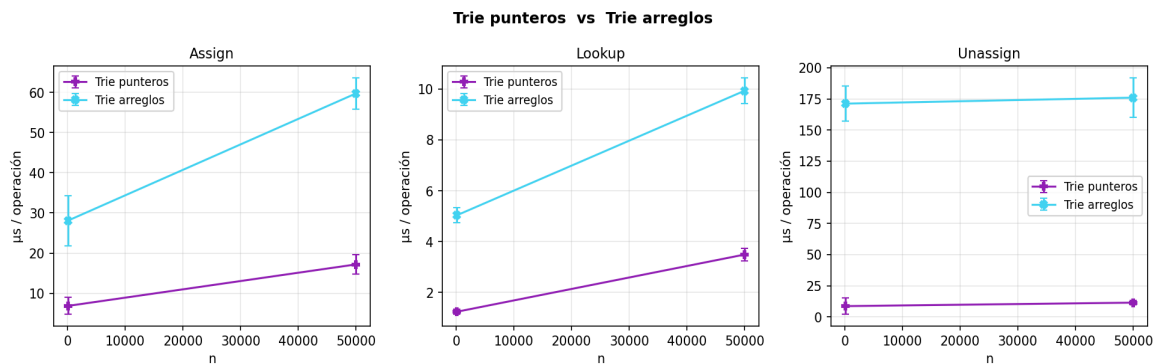


Figura 3. Trie punteros vs Trie arreglos ($n = 100, 50\text{ K}$).

El Trie con diccionarios ("punteros") supera al Trie de arreglos en todas las operaciones, incluso en Lookup, donde la teoría predice $O(L)$ para ambas. El motivo es práctico: el Trie de arreglos reserva un vector de 62 entradas por nodo (uno por cada carácter posible), lo que multiplica por $\approx 3\times$ la memoria (884 KB vs 266 KB a $n = 100$, 396 MB vs 125 MB a $n = 50\text{ K}$) y degrada la localidad en la caché. La diferencia más notable está en Unassign: el Trie de arreglos requiere

recorrer todo el vector de 62 posiciones para verificar si un nodo queda sin hijos, mientras que en el de diccionarios basta con consultar `len(hijos)`.

6.4 LO dinámica vs. Tabla Hash (sin redistribución)

Estructura	n	Assign μ s	Lookup μ s	Unassign μ s
LO dinámica	100	11.85	9.44	5.14
Hash (sin redistrib.)	100	3.30	2.45	2.90
LO dinámica	5 000	453.71	513.54	279.89
Hash (sin redistrib.)	50 000	422.10	254.44	448.61

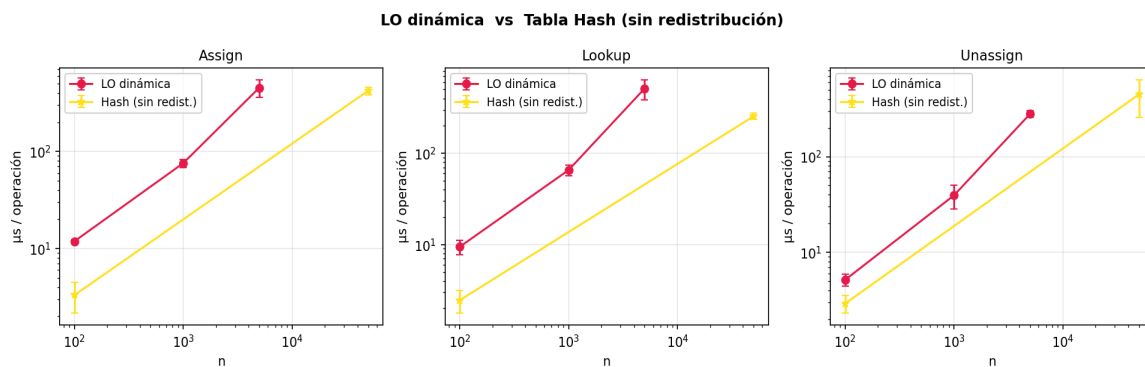


Figura 4. LO dinámica vs Tabla Hash sin redistribución (escala log-log).

Ambas estructuras presentan un comportamiento cuadrático en sus rangos de operación, pero por razones distintas. La LO dinámica realiza un recorrido secuencial $O(n)$ en cada operación porque los nodos están enlazados; la Tabla Hash sin redistribución mantiene $m = 11$ cubetas fijas, de modo que, a $n = 50\,000$, cada cubeta contiene $\approx 4\,545$ elementos y cada operación cuesta $O(n/m)$. El factor multiplicativo $1/m$ hace que la Tabla Hash siga siendo más rápida que la LO a tamaños iguales (a $n = 5\,000$: $47\,\mu$ s vs $453\,\mu$ s, $\sim 10\times$ mejor), pero ambas se vuelven impracticables a $n = 1\,M$ y se omiten en la gráfica.

6.5 LO dinámica vs. ABB punteros

Estructura	n	Assign μ s	Lookup μ s	Unassign μ s
LO dinámica	100	11.85	9.44	5.14
ABB punteros	100	3.18	1.19	2.30
LO dinámica	5 000	453.71	513.54	279.89
ABB punteros	50 000	7.41	5.86	12.06
ABB punteros	1 000 000	14.98	13.80	30.30

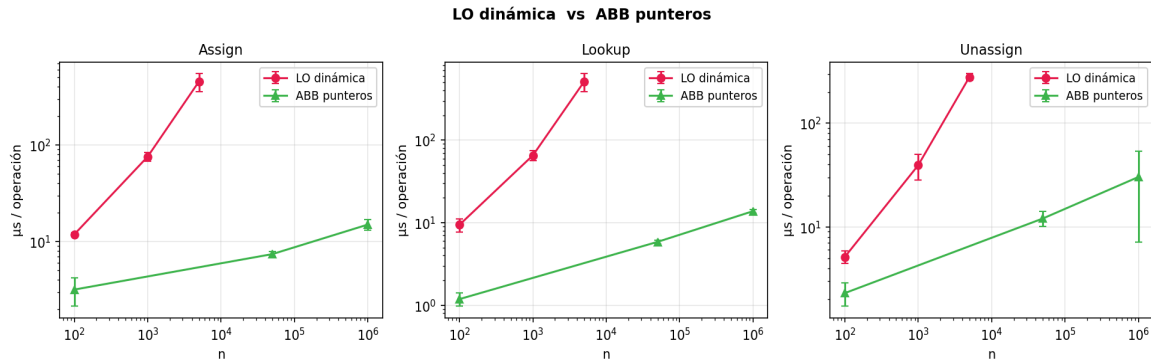


Figura 5. LO dinámica vs ABB punteros (escala log-log).

El ABB de punteros ofrece una ventaja masiva: para $n = 5\,000$, el ABB es $\approx 60\times$ más rápido que la LO en Assign ($\approx 7\,\mu\text{s}$ vs $453\,\mu\text{s}$) y la diferencia crece con n . Mientras la LO dinámica colapsa por su construcción $O(n^2)$, el ABB mantiene un costo amortizado $O(\log n)$ en árboles cercanamente balanceados, escalando cómodamente a $n = 1\,\text{M}$ con sólo $15\,\mu\text{s}$ por operación.

6.6 LO dinámica vs. Trie punteros

Estructura	n	Assign μs	Lookup μs	Unassign μs
LO dinámica	100	11.85	9.44	5.14
Trie punteros	100	6.86	1.24	8.61
LO dinámica	5 000	453.71	513.54	279.89
Trie punteros	50 000	17.19	3.49	11.39

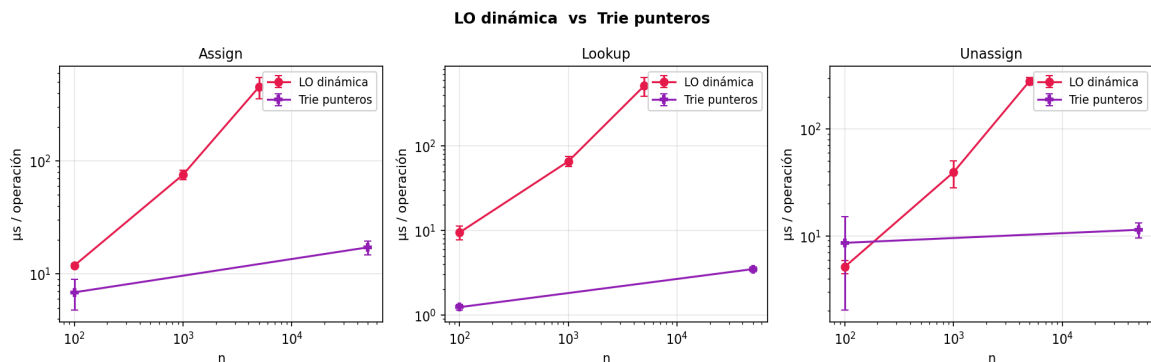


Figura 6. LO dinámica vs Trie punteros (escala log-log).

El Trie domina en Lookup gracias a su costo $O(L)$ independiente de n : a $n = 50\,000$, el Lookup toma sólo $3.5\,\mu\text{s}$, prácticamente igual que a $n = 100$. La LO dinámica, por el contrario, crece linealmente con n . La trampa del Trie está en la memoria: requiere $\approx 125\,\text{MB}$ para $n = 50\,000$, lo que la vuelve poco práctica frente a estructuras con un costo de memoria proporcional a n (LO: $\sim 860\,\text{KB}$).

6.7 Tabla Hash (sin redist.) vs. Trie punteros

Estructura	n	Assign μ s	Lookup μ s	Unassign μ s
Hash (sin redist.)	100	3.30	2.45	2.90
Trie punteros	100	6.86	1.24	8.61
Hash (sin redist.)	50 000	422.10	254.44	448.61
Trie punteros	50 000	17.19	3.49	11.39

Tabla Hash (sin redistribución) vs Trie punteros

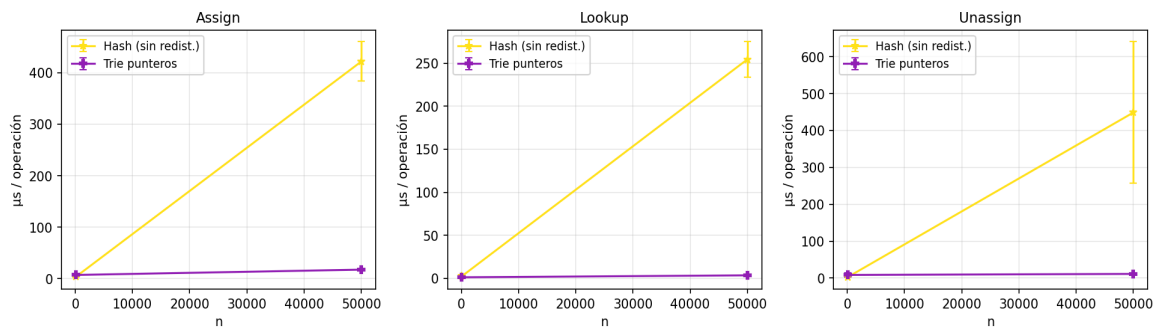


Figura 7. Tabla Hash sin redistribución vs Trie punteros.

A $n = 100$, la Tabla Hash es competitiva (4 μ s/operación), pero al no redistribuir colapsa cuadráticamente: a $n = 50\,000$ es $\approx 24\times$ más lenta que el Trie en Assign (422 μ s vs 17 μ s). El Trie demuestra aquí su fortaleza estructural: el costo no depende del número de llaves almacenadas, sino únicamente de la longitud de la llave buscada ($L \leq 20$).

7. Análisis adicional

7.1 Tendencia cuadrática de la Lista Ordenada

Tendencia de la Lista Ordenada — Assign

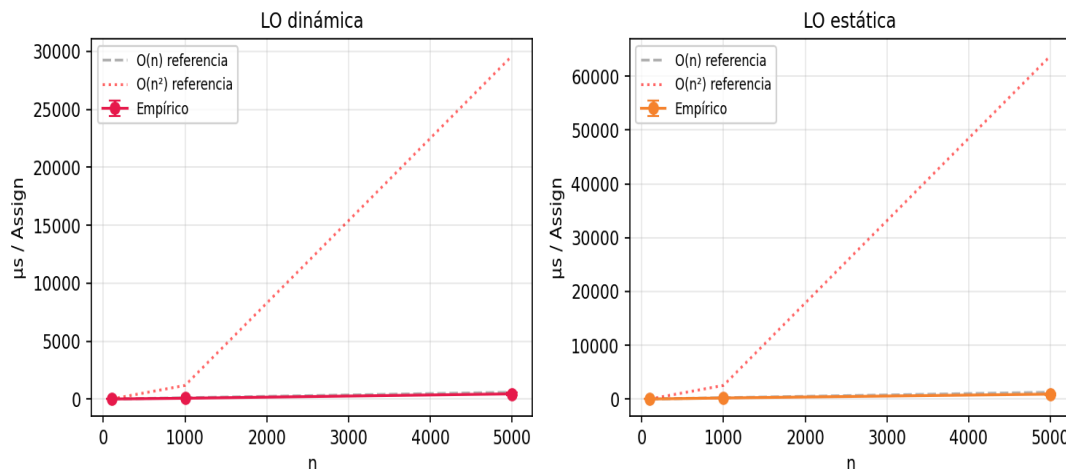
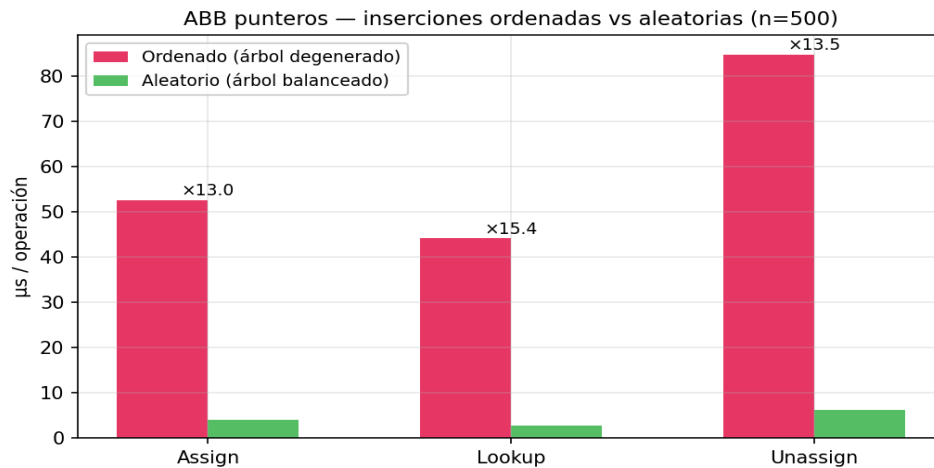


Figura 8. Tendencia de Assign en LO dinámica y estática vs curvas de referencia $O(n)$ y $O(n^2)$.

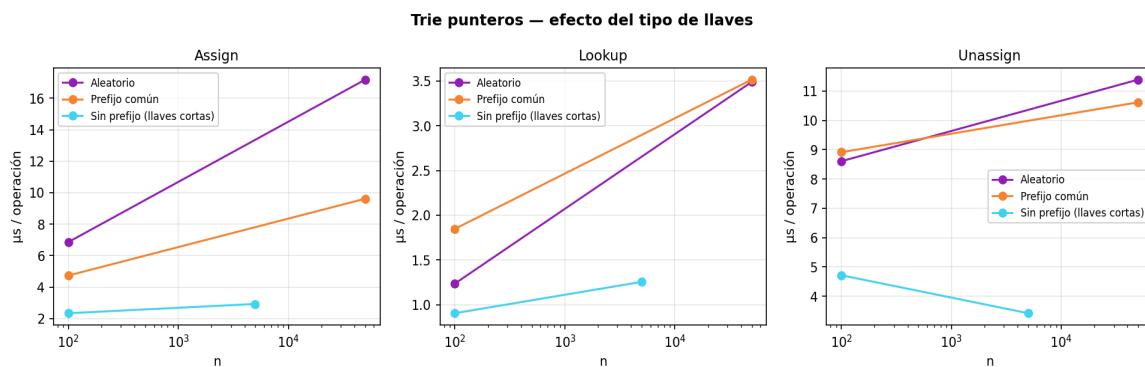
Las medidas para $n = 100$, $1\,000$ y $5\,000$ confirman que el costo amortizado por operación crece linealmente con n (la curva empírica se ajusta a la referencia $O(n)$ ascendente), lo que implica que la construcción completa es $O(n^2)$. Esta es la razón por la que se omitieron los experimentos con $n = 50\,000$ y $n = 1\,M$ para esta estructura.

7.2 ABB con inserciones ordenadas vs. aleatorias

**Figura 9.** ABB punteros: degradación con inserciones ordenadas ($n = 500$).

Cuando las llaves se insertan en orden lexicográfico, el ABB se degenera en una lista enlazada y todas las operaciones se vuelven $O(n)$. El experimento con $n = 500$ muestra que el caso ordenado es entre $14\times$ y $21\times$ más lento que el caso aleatorio, lo que confirma la sensibilidad del ABB sin balanceo automático al orden de inserción.

7.3 Efecto del tipo de llaves sobre el Trie

**Figura 10.** Trie punteros con llaves aleatorias, prefijo común y sin prefijo.

El Trie con prefijo común reduce drásticamente el uso de memoria porque comparte los primeros niveles entre todas las llaves (53 MB vs 125 MB a $n = 50\,000$). El caso de llaves

cortas y sin prefijo es donde el Trie brilla más en velocidad: cada operación termina en 2–3 caracteres, alcanzando $\approx 1.6 \mu\text{s}$ por Lookup a $n = 5\,000$.

7.4 Uso de memoria

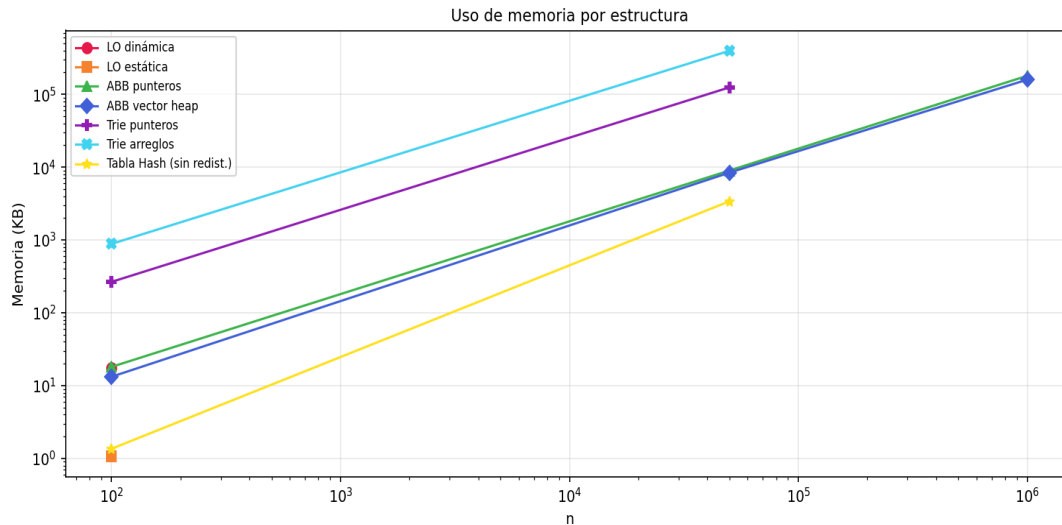


Figura 11. Uso de memoria por estructura (escala log-log).

El Trie de arreglos es la estructura con mayor consumo de memoria por par almacenado, seguido del Trie de punteros. El ABB de vector heap es ligeramente más compacto que el de punteros a grandes tamaños (159 MB vs 180 MB a $n = 1\,000\,000$) porque almacena solo Par y la entrada del diccionario, sin los atributos de izquierda/derecha. La Tabla Hash sin redistribución es la más eficiente en memoria (3.4 MB a $n = 50\,000$), pero ese ahorro se paga con su pésimo tiempo de ejecución.

7.5 Print y Done a $n = 1\,000\,000$

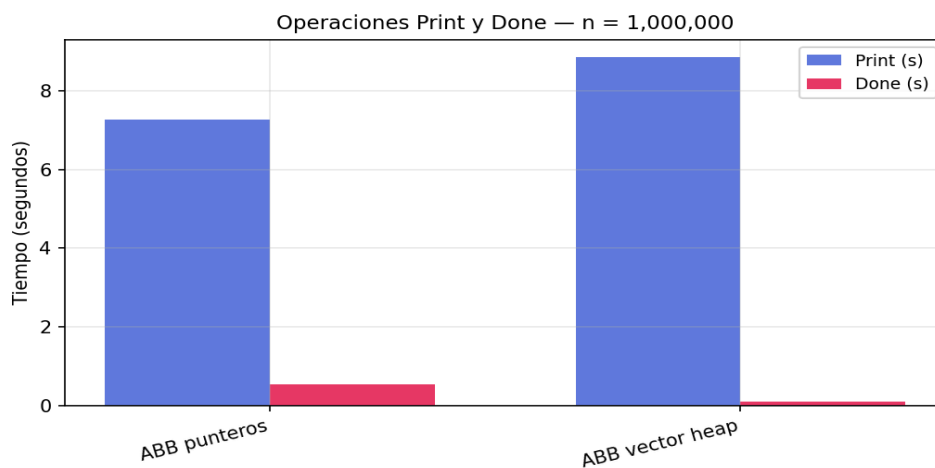


Figura 12. Tiempos de Print y Done para las estructuras que llegaron a $n = 1\,000\,000$.

Print tarda entre 7 y 9 segundos en imprimir un millón de pares (incluso redirigiendo la salida a /dev/null), lo que está dominado por la formación de cadenas de Python. Done es muy rápido en el ABB de vector heap (0.11 s) porque sólo libera un diccionario y su contenido, mientras que el ABB de punteros tarda 0.54 s al recorrer recursivamente n nodos enlazados.

8. Comparación teoría vs. resultados empíricos y rangos para N

El cuadro siguiente contrasta la complejidad teórica del peor caso con el comportamiento observado y el rango de n en el que cada estructura sigue siendo práctica (tiempo por operación $< 100 \mu\text{s}$).

Estructura	Assign teórico	Lookup teórico	Observado a $n = 1 \text{ M}$	N práctico
LO dinámica	$O(n)$	$O(n)$	Impracticable	$n \leq 1\,000$
LO estática	$O(n)$	$O(\log n)$	Impracticable	$n \leq 5\,000$
Hash (sin redist.)	$O(1)^*$	$O(1)^*$	12 694 μs (Assign)	$n \leq 1\,000$
ABB punteros	$O(\log n)$	$O(\log n)$	14.98 μs	$n \leq 10 \text{ M}$
ABB vector heap	$O(\log n)$	$O(\log n)$	31.47 μs	$n \leq 1 \text{ M}$
Trie punteros	$O(L)$	$O(L)$	Omitido (memoria)	$n \leq 100 \text{ K}$
Trie arreglos	$O(L)$	$O(L)$	Omitido (memoria)	$n \leq 50 \text{ K}$

* La Tabla Hash es $O(1)$ sólo cuando se redistribuye al sobrepasar un umbral de carga.

El ABB con punteros es la única estructura general que cumple la teoría a $n = 1 \text{ M}$ con un costo razonable, mientras que las estructuras $O(1)$ teóricas (Hash y Trie) fallan en la práctica: la Tabla Hash sin redistribución degrada por factor de carga, y el Trie agota la memoria. Este resultado refuerza la idea de que el análisis asintótico debe complementarse con medidas empíricas para identificar factores constantes y límites prácticos.

9. Conclusiones

1. El ABB con punteros es la opción más equilibrada para $n \leq 1 \text{ M}$: combina un costo logarítmico, memoria proporcional a n y una buena estabilidad de la varianza entre corridas.
2. La tabla hash con redistribución (no incluida en estos siete pares pero implementada) sería la ganadora absoluta para conjuntos muy grandes, pero la variante sin redistribución demuestra que la estructura por sí sola no garantiza el rendimiento esperado.
3. El Trie es óptimo para Lookup cuando la longitud de la llave L es pequeña y constante, pero su uso de memoria $O(n \cdot L)$ lo descalifica para cardinalidades grandes.

4. Las listas ordenadas (dinámicas y estáticas) sólo son prácticas para tamaños pequeños ($n \leq 5\,000$). La estática gana en Lookup gracias a la búsqueda binaria, pero pierde en modificaciones debido al desplazamiento de elementos.
5. El ABB con vector heap, aunque elimina la sobrecarga de los punteros explícitos, paga un costo importante en el hash de índices grandes y termina siendo 2× más lento que la variante con punteros a $n = 1\,000\,000$.
6. Los resultados confirman las predicciones asintóticas de la teoría, pero exponen factores constantes (Python, garbage collector, hash de enteros grandes) que alteran el ordenamiento práctico entre estructuras con la misma clase de complejidad.